Code-Switching Translation with PEFT on a Frozen Qwen3-1.7B

Midterm Project · Advanced Topics in Large Language Models · 2026xxxxxx · (name)

1. Overview

We build two parameter-efficient translation systems on a frozen **Qwen3-1.7B** backbone that translate single-language or code-switched English/Spanish/Korean/Japanese input into English or Korean. Both systems are LoRA adapters on all linear projections of all 28 layers: the **5M system** uses rank 4 (4,358,144 trainable parameters) and the **50M system** rank 44 (47,939,584). Because no code-switched translation corpus exists for these languages, we synthesise the training data reference-first: an LLM writes clean sentences with faithful four-language versions and two code-switched variants whose matrix language, embedded language(s) and switching pattern we control explicitly (noun insertion with native particles, clause alternation, phrase mixing, light-verb constructions); variants are verified by an English-pivot round trip with an independent local model. On our development set (1,535 items, built like the official validation data) the untuned baseline scores 53.98; the 5M system reaches 73.43 and the 50M system 74.54, with the 50M system ahead on every subset.

2. PEFT design

Both systems add LoRA adapters (Hu et al., 2021) to **every linear projection of every decoder layer** of the frozen Qwen3-1.7B backbone (28 layers; q/k/v/o projections of the attention block and gate/up/down projections of the MLP), with `lora_alpha = 2r`, no dropout, no bias, and the backbone weights never updated. Qwen3-1.7B has hidden size 2048, MLP size 6144 and 8 KV heads of dimension 128, so one rank of LoRA on all seven projections of one layer costs 4096+3072+3072+4096+8192+8192+8192 = 38,912 parameters, i.e. 1,089,536 parameters per unit of rank for the whole network.

System	Rank r	alpha	Modules	Trainable params	Budget use
5M	4	8	q,k,v,o,gate,up,down × 28 layers	4,358,144	87.2%
50M	44	88	q,k,v,o,gate,up,down × 28 layers	47,939,584	95.9%

Motivation. Under a fixed parameter budget the literature consistently favours spreading low-rank adapters over all linear layers (in particular the MLP) rather than concentrating a high rank on attention only (LoRA module ablations; QLoRA's "all linear layers" finding; Biderman et al., 2024). r=4 is the largest uniform all-linear rank that fits 5M, and r=44 the largest that fits 50M, so the 50M system is a strict superset of the 5M system: same modules, same data, same recipe, 11× the rank. We also tried DoRA at r=4 (4,931,584 parameters) and a learning-rate sweep; see Section 6.

Training. Supervised fine-tuning on (prompt, translation) pairs rendered with the Qwen3 chat template in non-thinking mode, i.e. the prompt ends with `<|im_start|>assistant\n<think>\n\n</think>\n\n` exactly as at inference, and the loss is computed only on the translation tokens plus `<|im_end|>`. AdamW, cosine schedule, 3% warm-up, effective batch 16, bf16, gradient checkpointing; one RTX 3080 (12 GB) trains ~2,300 tokens/s.

3. Training data generation

No suitable code-switched translation corpus exists for en/es/ko/ja, so all training data is synthetic and built **reference-first**: a clean target-language sentence is written first and the code-switched input is derived from it, so the reference never contains generation noise and stays literal, which is what a single-reference chrF++ rewards.

Step 1 – scenarios. 48 domains (business meetings, finance, hospital, pharmacy, airports, hotels, restaurants, IT operations, logistics, customs, education, HR, legal, manufacturing, retail, ...) × 30 concrete situations with two participant roles each (1,440 scenarios), generated by an LLM (Claude Sonnet via the headless `claude -p` CLI; the same driver works with a local model through vLLM, `scripts/gen_vllm.py`).

Step 2 – 4-way tuples with controlled switching. For each scenario the LLM writes 8 different sentences (7–30 words, varied speech acts, concrete dates/amounts/names, no idioms), each with a faithful literal English, Korean (polite -습니다/-세요 register), Spanish and Japanese (です/ます) version, plus **two code-switched variants**. Every variant follows a per-sentence specification `matrix=X; embed=Y[,Z]; pattern=P` sampled by us (matrix language ko 35% / en 25% / ja 20% / es 20%; one embedded language 70%, two 30%; pattern *noun* 50%, *clause* 22%, *phrase* 18%, *verb* 10%). The matrix language supplies grammar, word order and all particles; embedded spans are copied verbatim from the parallel sentence. The patterns mirror the specification's examples: noun/NP insertion with native particles attached ("chest pain이", "meetingが", "El informe의 마지막 section을"), clause alternation at a clause boundary ("Aunque el tren llegó tarde, 우리는 회의에 늦지 않았습니다."), phrase mixing, and Korean/Japanese light-verb constructions ("주문을 cancel하고", "confirmしてください"). This design follows the linguistic evidence that natural ko–en / ja–en code-switching is predominantly insertional (nouns with matrix-language morphology) and that LLM generation is only reliable with explicit constraints.

Step 3 – filtering. (i) script checks: Korean fields must be Hangul-only, Japanese fields must contain kana/kanji, and the English/Spanish fields must be identified as English/Spanish by a language identifier (this removed ~2% of tuples whose en/es fields were swapped); (ii) each mixed variant must actually contain ≥2 scripts (or en+es by LID); (iii) an **English-pivot round trip**: every mixed variant is back-translated into English by an independent local model (Qwen3-14B-AWQ, vLLM) and variants with chrF++ < 40 against the tuple's English sentence are dropped (≈2.4%); (iv) exact de-duplication and removal of any sentence overlapping the development set.

Step 4 – assembling rows. Each surviving mixed variant yields two rows (→ English, → Korean, references = the tuple's en / ko sentences); the four monolingual versions yield single-language rows (es/ja/ko → en, en/es/ja → ko), sub-sampled so that ~30% of the rows are single-language; ~5% of tuples also contribute an identity row (en→en or ko→ko) so the model learns to copy text already in the target language. The final set has 26,177 clean tuples → 147,252 synthetic rows (mixed→en 51,538; mixed→ko 51,539; single→en 22,227; single→ko 21,948, of which 388 identity rows), plus 40,000 human-translated sentence pairs (AI Hub ko–en daily/general/specialised and ko–ja, BSD business dialogues ja–en, Tanaka ja–en, TED2020 es→ko/es→en/ja→ko/ko→en; see Section 5.2 for why) = 187,252 rows. Mixed rows cover all six language pairs (10–15k each) and all four triples (6.5–8k each); 28% of mixed rows use three languages.

Development set. Because the official validation data was not yet available, we built our own: 384 tuples generated with a different prompt and different scenario seeds (and, as a robustness check, a second set generated by a different model family, Qwen3-8B), giving 1,535 items: mixed→en 384, mixed→ko 383, single→en 384, single→ko 384. All model selection used this set.

4. Inference design

The instruction is the default template from the specification (target language inserted), wrapped in the Qwen3 chat template with `enable_thinking=False`; no few-shot examples, no system prompt. Decoding is greedy (`do_sample=False`), at most 256 new tokens, stopping on `<|im_end|>`; the adapter is applied unmerged with HF transformers (we found vLLM's LoRA path changes ~25% of greedy outputs, so all reported numbers and the submitted predictions use the same HF path). Predictions are NFC-normalised and stripped.

5. Results

All numbers are corpus-level chrF++ (sacrebleu `CHRF(word_order=2)`) on our development set (Section 3) under the official aggregation: single = mean(single→en, single→ko), mixed = mean(mixed→en, mixed→ko), score = 0.3·single + 0.7·mixed. The baseline is the untuned Qwen3-1.7B with the default instruction template, greedy, non-thinking.

5.1 Main results

System	params	single→en	single→ko	mixed→en	mixed→ko	single	mixed	score	dev2

Baseline (no PEFT)	0	66.08	31.85	72.74	39.50	48.97	56.12	53.98	68.82
5M (LoRA r=4)	4,358,144	78.79	58.19	83.79	67.31	68.49	75.55	**73.43**	73.24
50M (LoRA r=44)	47,939,584	79.98	60.51	83.99	68.76	70.25	76.38	**74.54**	71.41
Copy input	–	7.38	2.20	29.86	20.30	4.79	25.08	18.99	–

Both systems clear the baseline by a wide margin (+19.5 and +20.6 points), and the 50M system is ahead of the 5M system on every one of the four subsets (Success criterion 2), with the largest margins on Korean targets (+2.3 single→ko, +1.5 mixed→ko). The gain over the baseline is largest for Korean targets (+26 to +29 points) because that is where the untuned model failed outright. Both systems were trained for 2 epochs; the epoch-1 checkpoints scored 72.27 (5M) and 72.87 (50M).

5.2 How the score grew with data

training tuples (rows)	5M	50M
0 (baseline)	53.98	53.98
447 (2.5k rows, pilot)	62.16	63.18
6.3k (35.6k rows, round 1)	70.19	71.70
6.3k + 30k human pairs	70.78	71.41
26.2k + 40k human (187k rows)	73.43	74.54

Even 447 synthetic tuples lift the score by 8 points, mostly by removing the baseline's Korean-side failures (instruction echoing, repetition loops, answering in the wrong language: 4–5% of Korean-target outputs at baseline, ≈0% after fine-tuning). Adding 30k human-translated sentence pairs (AI Hub ko–en/ko–ja, BSD business dialogues, Tanaka, TED2020) on top of the synthetic data gives a further +0.6 on our dev set and +0.8 on the second dev set, so the final systems include them.

5.3 Robustness check on a second dev set

Because our dev set was generated by the same model family as the training data (Claude Sonnet), we also score a second set built from Qwen3-8B tuples (1,267 items). Baseline 68.82 → final 5M 73.24, final 50M 71.41 (72.54 after its first epoch). The 5M/50M order is reversed on this set: its Korean references were written by a weaker model and follow Qwen3-8B's style, and the higher-capacity adapter fits the Claude-style references of the training data more tightly (its dev2 score falls from epoch 1 to epoch 2 while the main dev score rises). We therefore treat dev2 only as a robustness check: the ordering on the main dev set, which matches the specification's examples in style, is 50M > 5M on every subset, and the official validation set should be used to confirm it.

5.4 Additional controlled comparisons (50M budget, same data)

50M variant	params	single→en	single→ko	mixed→en	mixed→ko	score	dev2
uniform r=44 (submitted)	47,939,584	79.98	60.51	83.99	68.76	**74.54**	71.41
MLP-heavy: attn r=20 / MLP r=60	49,315,840	79.12	61.40	83.61	68.71	74.39	72.37
uniform r=44 + dropout 0.05, wd 0.01	47,939,584	79.34	60.64	83.77	68.31	74.23	71.38
uniform r=44, 3rd epoch (lr 8e-5)	47,939,584	79.06	60.49	83.44	68.96	74.27	70.90

Re-allocating rank towards the MLP, adding regularisation, or training a third epoch all land within ±0.3 of the submitted configuration on the main dev set (the MLP-heavy variant is more robust on dev2 but falls 0.2 below the 5M system on mixed→en, which would break the "50M ≥ 5M on every metric" requirement). A third epoch for the 5M system likewise gave 73.43 (no change). Together with the pilot sweeps this suggests that, at this data scale, the remaining headroom is in the data (reference style and coverage) rather than in the adapter configuration.

6. Analysis

What the adapters fix. The baseline's low Korean-target scores come mostly from catastrophic outputs rather than weak translation: it echoes or translates the instruction instead of the text, falls into

repetition loops, or answers in the wrong language (4–5% of Korean-target items; e.g. for "You have five remaining vacation days this year, ..." it emits "이해를 위해, 이 문장은 다음과 같이 번역됩니다." repeated until the token limit). After fine-tuning these disappear (wrong-language outputs: 33 of 767 Korean-target items at baseline → 5 of 767 for the final 5M system; outputs with length ratio < 0.75: 36 → 13 of 1,535; exact matches with the reference: 29 → 119; items below 45 chrF++: 600 → 143; no empty outputs), and the model reliably copies the parts of the input that are already in the target language while translating the rest, e.g. "Please send me the 장학금 신청서를 by October 31, 그리고 어떤 서류를 첨부해야 하는지 알려 주세요." → "10월 31일까지 장학금 신청서를 보내 주시고, 어떤 서류를 첨부해야 하는지 알려 주세요." (exact match; baseline 8 chrF++).

Switch-boundary handling. Clause-level alternation and particle-attached insertions are handled well, including three-language inputs: "Could you tell me which 書類を持っていけばよいか, such as my passport, 病院の在職証明書, and two photos?" → "여권, 병원 재직증명서, 사진 두 장이면 어떤 서류를 가져가야 하는지 알려 주시겠습니까?" (87). Mixed inputs with three languages score a few points below two-language inputs (ko target: 61.7 vs 65.0 chrF++ for the round-1 50M system), and Korean targets remain ~18 points below English targets for both single and mixed inputs, i.e. the difficulty is the Korean generation side, not the switch detection.

Remaining failures (items below 45 chrF++: 168 of 1,535 for the round-1 50M system, 159 of them with Korean target) are dominated by lexical and register choices rather than structural errors: "The final exam은 multiple-choice 40문항과 essay 두 개이고, we will have two hours to finish." → reference "기말시험은 객관식 40문항과 서술형 2문항이며, 우리에게는 끝내는 데 2시간이 주어집니다." vs output "최종 시험은 40문항의 단체형 문제와 자유형 문제 두 개로, 두 시간 동안 풀 수 있습니다." — the meaning is largely preserved but "단체형/자유형" are wrong terms for multiple-choice/essay. True meaning changes concentrate around Japanese-heavy inputs with English nouns inside Japanese grammar ("Will the reading responseの deadlineは、Tuesday sectionではなく Thursday sectionに出席するstudentsでも同じですか" → output garbles "Tuesday section/Thursday section" into "화요일 쪽으로 Thursday 쪽에"), where the 1.7B backbone's Japanese comprehension is the bottleneck. Because chrF++ is recall-weighted, the model's tendency to stay literal and complete (length ratio ≈1.0 on every subset) is the right behaviour for this metric.

Budget comparison. The 50M system beats the 5M system on all four subsets of the main dev set at every data size we tried (Table in 5.2), with the largest margin on Korean targets (+2.5 mixed→ko, +2.6 single→ko in round 1), consistent with the view that r=4 is capacity-limited for the Korean generation side. A learning-rate sweep (1e-4 / 3e-4 / 6e-4) and DoRA at r=4 changed the pilot score by less than ±0.3, so the final systems use plain LoRA (5M: lr 3e-4, 50M: lr 2e-4, cosine, 3% warm-up, effective batch 16).